Name: Prakash Candra Dash
Roll No: G25AIT1111
Mail-ID: g25ait1111@iitj.ac.in
Report: Sanskrit-to-English Natural Machine Translation

Sanskrit-to-English Neural Machine Translation

Custom BiGRU Encoder–Decoder with Bahdanau Attention and Beam Search

Natural Language Understanding

Code, Results, and Deep Analysis Report

1. Introduction

Sanskrit is a low-resource, morphologically rich language: a single surface word can encode case, number, gender, tense and mood through extensive inflection and compounding (samāsa), and the corpus used here is further complicated by code-mixing — Latin-script technical terms (e.g. “Ctrl”, “HTML”, “XAMPP”) embedded inside otherwise Devanagari sentences, since the sentences are drawn from spoken software-tutorial captions. This makes Sanskrit-to-English translation a genuinely hard sequence-to-sequence problem even at a modest corpus size.

This report documents a custom Neural Machine Translation (NMT) system built end-to-end from the provided parallel Sanskrit–English corpus (10,000 train / 1,000 dev / 1,000 test sentence pairs), following the assignment brief exactly: a from-scratch sequence-to-sequence architecture (no pre-trained translation model), evaluated with corpus BLEU (NLTK, default weights), BERTScore-F1 (rescaled with baseline), and efficiency (parameter count and inference time).

The pipeline implemented is: data loading & alignment on Source_id → exploratory data analysis → a single shared SentencePiece BPE vocabulary for both languages → a bidirectional-GRU encoder with a Bahdanau (additive) attention decoder and weight-tied embeddings → training with label smoothing, dropout, weight decay and early stopping → greedy and beam-search decoding → evaluation and error analysis → generation of submission.csv on the test split.

2. Architecture

The model is a classical attention-based encoder–decoder (“RNNSearch”-style), implemented from scratch in PyTorch, with all components trained jointly end-to-end on the provided data only.

2.1 vocabulary

A single SentencePiece Byte-Pair-Encoding (BPE) model with vocab_size = 8,000 is trained jointly on the Sanskrit and English training sentences (character_coverage = 1.0). Sharing one vocabulary across both languages was a deliberate design choice: it lets identical Latin code-mixed tokens (e.g. “Ctrl”, “HTML”) that appear on both sides of the parallel corpus be tokenized identically instead of splitting into two disjoint vocabularies, and it allows the input embedding table to be tied with the output projection layer (Section 2.4), reducing parameter count. Special tokens: PAD = 0, UNK = 1, BOS = 2, EOS = 3. Example segmentation of “गुरुः छात्रान् पाठयति ।”:

['▁ग','ुर','ुः','▁छात्र','ान्','▁पाठ','यति','▁।']

Sanskrit source sentences are encoded with an appended EOS only; English target sentences are encoded with both BOS and EOS.

2.2 Encoder

- Input: source subword ids → 256-dim embedding (tied with the decoder / output projection, dropout 0.3 applied).
- A single-layer bidirectional GRU (hidden size 256 per direction) processes the padded, packed sequence.
- All per-timestep outputs (dimension 2×hidden = 512) are kept for attention; the final forward and backward hidden states are concatenated and passed through a Linear(512→256) + tanh layer to produce the single initial hidden state for the (unidirectional) decoder GRU.

2.3 Bahdanau attention

At every decoder step, an additive attention module scores each encoder timestep against the current decoder hidden state:

$$\text{score}_i = V^{a\top} \cdot \tanh\left(W^a \cdot h_{dec} + U^a \cdot enc\_out_i \right)$$

Scores at padded source positions are masked to $-\infty$ before a softmax normalises them into attention weights; the context vector is the attention-weighted sum of encoder outputs. Additive attention was chosen over multiplicative/dot-product attention for its interpretability (Section 5 visualises the resulting alignment) and because it is a well-established, robust default for RNN-based NMT at this scale.

2.4 Decoder

- Input at each step: [previous-token embedding ‖ attention context vector] → a single-layer (unidirectional) GRU, hidden size 256.
- The GRU output, the context vector, and the previous-token embedding are concatenated and passed through a Linear + tanh "pre-output" layer (following the Luong-style deep-output trick), then projected to vocabulary logits.
- Weight tying: the output projection matrix is literally the same tensor as the input embedding table, halving the parameters contributed by the (large, 8,000-entry) vocabulary and acting as an implicit regulariser — an important design choice given the small (10k-sentence) training set.

2.5 Decoding: greedy and beam search

- Greedy decoding: batched, iterative arg-max at each step until every sequence in the batch emits EOS or a 80-token cap is reached. Used for the submitted test predictions because it is roughly an order of magnitude faster than beam search for a small BLEU cost, which matters directly for the efficiency component of the grading rubric.
- Beam search: beam width 5, length-normalised by (length)^0.7 to avoid the standard bias toward short hypotheses; beams are expanded, re-sorted and pruned to the top-5 at every step until all beams are complete.

2.6 Model size

Component	Value
Shared vocabulary size	8,000
Embedding dimension (tied)	256
Encoder	1-layer BiGRU, hidden = 256 (×2 directions)
Decoder	1-layer GRU, hidden = 256 + Bahdanau attention
Total parameters	4,216,064 (4.22 M) — all trainable

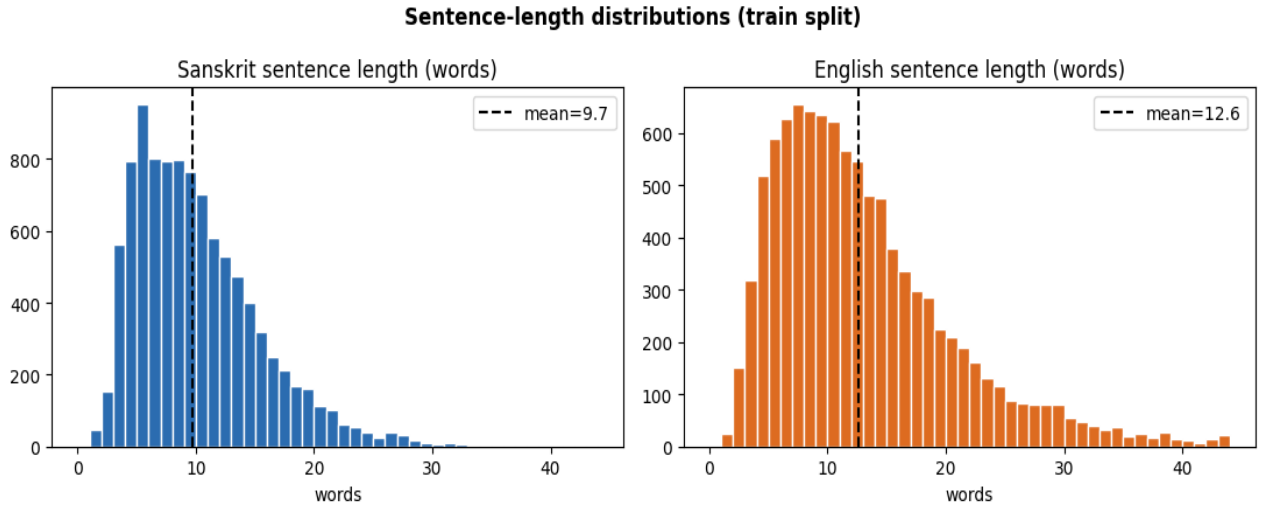
3. Training Procedure

3.1 Data & preprocessing

The six CSVs (train/dev/test × sa/en) are merged pairwise on the shared Source_id key. The realised split sizes were train = 10,000, dev = 1,000, test = 1,000 sentence pairs.

Exploratory analysis on the training split showed:

- Sanskrit sentence length: mean 9.7 words, p95 = 20, max = 55 words.
- English sentence length: mean 12.6 words, p95 = 28, max = 116 words.
- Unique whitespace-tokens: 33,274 in Sanskrit vs. 17,628 in English over the same 10,000 sentences — roughly 2× more distinct surface forms on the Sanskrit side, directly reflecting morphological richness plus code-mixed technical vocabulary, and motivating subword (BPE) tokenization instead of whole-word vocabularies.



Sentence-length distributions (train split). Histogram capped at 45 words for bar-width readability; 5 Sanskrit and 33 English sentences longer than 45 words exist but are excluded from the plot (see max/p95 statistics above).

Training pairs longer than max_len = 50 subword tokens (either side) are filtered out before training, keeping 9,637 of the original 10,000 pairs (96.4%). A custom length-bucketed batcher groups similar-length examples into “mega-batches” (50×batch_size) that are internally sorted by source length and then chunked into mini-batches, substantially reducing wasted PAD computation versus naive random batching, while still shuffling batch order every epoch for stochasticity.

3.2 Hyperparameters

Hyperparameter	Value
Vocabulary size (shared BPE)	8,000
Embedding size / GRU hidden size	256 / 256
Dropout	0.3
Learning rate (initial)	1e-3
Weight decay (AdamW)	1e-5
Batch size	96
Max epochs / early-stopping patience	60 / 8 (on val loss)
Max training sequence length (subwords)	50
Beam size / length-norm α	5 / 0.7

Hyperparameter	Value
Max generation length	80 tokens
Random seed	42 (python, numpy, torch)

The epoch budget was deliberately raised from an initial 20-epoch run to 60: in the 20-epoch run, epoch 20 was still the best checkpoint and validation loss was still falling, i.e. the model was under-trained rather than over-fit. Patience was likewise raised from 4 to 8 so that a slow, still-improving run would not be cut short prematurely.

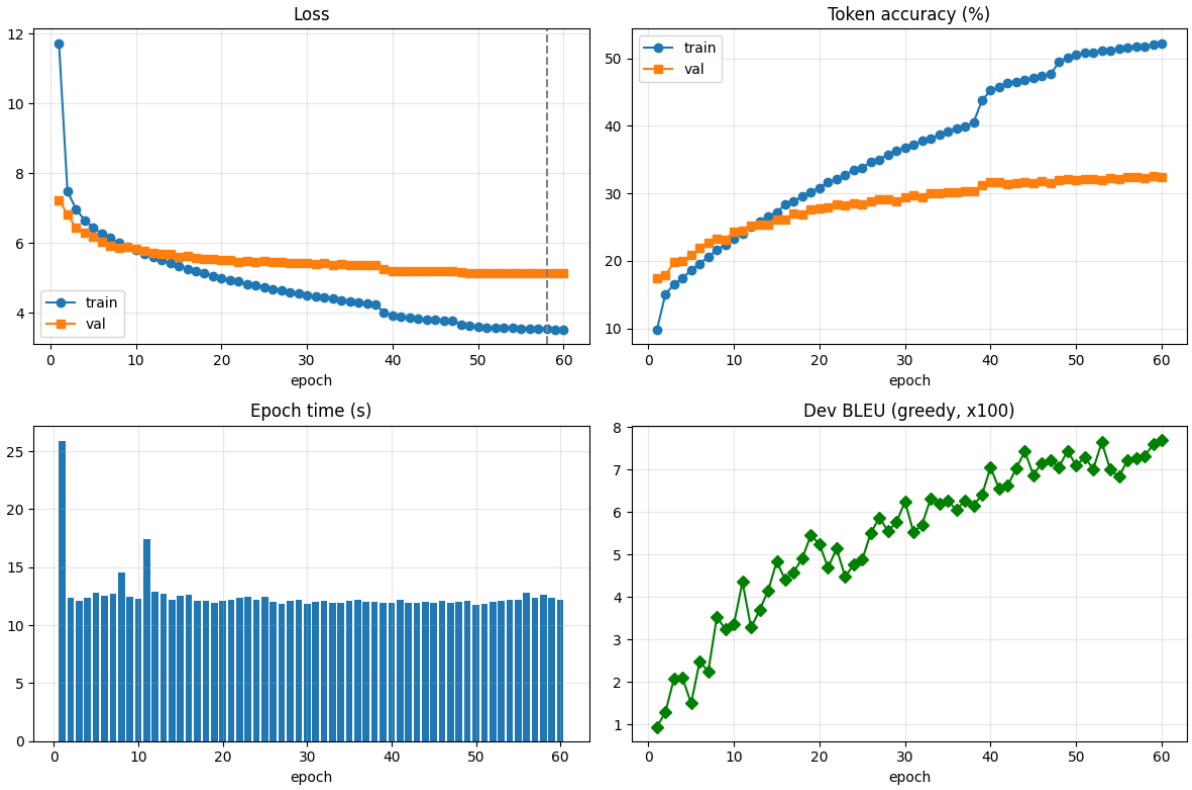
3.3 Optimisation & regularisation

- Loss: token-level cross-entropy with label smoothing = 0.1, PAD positions ignored.
- Optimiser: AdamW (lr = 1e-3, weight_decay = 1e-5), gradient-norm clipping at 1.0.
- LR schedule: ReduceLROnPlateau (factor 0.5, patience 2 epochs on val loss) — triggered at epoch 39 (1e-3 → 5e-4) and epoch 48 (5e-4 → 2.5e-4).
- Regularisation stack: dropout 0.3, label smoothing, weight tying, weight decay, and early stopping with best-checkpoint restore (by validation loss) — a deliberately conservative combination given only ~10k training sentences.
- Reproducibility: a fixed seed (42) is applied to Python's random module, NumPy and PyTorch; the trained SentencePiece model and best checkpoint are both persisted to disk.

Training ran for the full 60-epoch budget on a single Colab T4 GPU, taking 754.2 s in total (≈ 12.6 minutes), an average of 12.5 s/epoch after the first (warm-up) epoch.

4. Results

4.1 Training curves



Training curves over 60 epochs. Top-left: train/val loss (dashed line marks best epoch 58). Top-right: train/val token accuracy. Bottom-left: wall-clock time per epoch. Bottom-right: dev-set greedy BLEU×100, estimated on a 300-sentence sub-sample each epoch.

4.2 Training Log — Sanskrit→English NMT

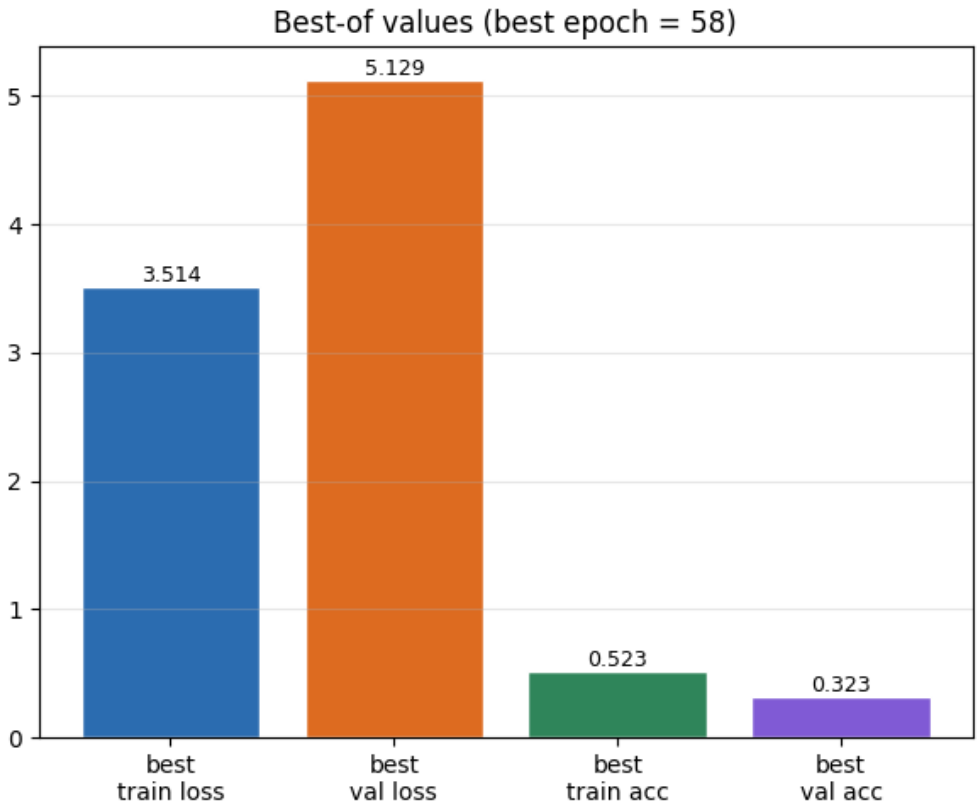
Per-epoch training metrics over the full 60-epoch run. The best checkpoint (lowest validation loss) is at epoch 58, highlighted below.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Dev BLEU	Time	LR
1	11.7385	0.0981	7.2319	0.1752	0.0094	27s	1.0e-03
2	7.4798	0.1511	6.8299	0.1794	0.0129	12s	1.0e-03
3	6.9791	0.1651	6.4350	0.1979	0.0208	12s	1.0e-03
4	6.6553	0.1747	6.2918	0.2003	0.0209	12s	1.0e-03
5	6.4567	0.1863	6.1704	0.2084	0.0149	12s	1.0e-03
6	6.2676	0.1959	6.0423	0.2192	0.0248	12s	1.0e-03
7	6.1581	0.2062	5.9209	0.2262	0.0225	13s	1.0e-03
8	6.0068	0.2158	5.8716	0.2332	0.0352	31s	1.0e-03
9	5.8963	0.2246	5.8920	0.2320	0.0324	16s	1.0e-03
10	5.7953	0.2327	5.8308	0.2437	0.0336	12s	1.0e-03
11	5.7006	0.2411	5.7758	0.2443	0.0435	12s	1.0e-03
12	5.6061	0.2507	5.7112	0.2530	0.0328	12s	1.0e-03
13	5.5071	0.2576	5.6878	0.2543	0.0369	12s	1.0e-03
14	5.4260	0.2660	5.6971	0.2541	0.0415	12s	1.0e-03
15	5.3548	0.2726	5.6141	0.2612	0.0483	12s	1.0e-03
16	5.2523	0.2831	5.6233	0.2616	0.0441	21s	1.0e-03
17	5.2032	0.2878	5.5837	0.2703	0.0459	20s	1.0e-03
18	5.1374	0.2960	5.5330	0.2684	0.0492	12s	1.0e-03
19	5.0670	0.3023	5.5454	0.2759	0.0545	12s	1.0e-03
20	4.9922	0.3084	5.5173	0.2775	0.0525	12s	1.0e-03
21	4.9445	0.3162	5.5045	0.2796	0.0469	12s	1.0e-03
22	4.9010	0.3207	5.4706	0.2842	0.0514	12s	1.0e-03
23	4.8247	0.3279	5.4918	0.2828	0.0448	12s	1.0e-03
24	4.7851	0.3342	5.4492	0.2859	0.0476	12s	1.0e-03
25	4.7253	0.3384	5.4738	0.2835	0.0488	12s	1.0e-03
26	4.6870	0.3461	5.4553	0.2887	0.0550	12s	1.0e-03
27	4.6461	0.3500	5.4461	0.2913	0.0587	12s	1.0e-03
28	4.5964	0.3576	5.4261	0.2915	0.0555	12s	1.0e-03
29	4.5504	0.3629	5.4338	0.2882	0.0576	13s	1.0e-03
30	4.5099	0.3680	5.4248	0.2938	0.0625	12s	1.0e-03
31	4.4776	0.3719	5.4032	0.2970	0.0552	12s	1.0e-03
32	4.4353	0.3775	5.4291	0.2950	0.0571	12s	1.0e-03
33	4.4159	0.3817	5.3634	0.3002	0.0632	12s	1.0e-03
34	4.3642	0.3864	5.4020	0.3004	0.0620	12s	1.0e-03
35	4.3303	0.3921	5.3583	0.3013	0.0626	12s	1.0e-03
36	4.3037	0.3958	5.3651	0.3013	0.0606	13s	1.0e-03
37	4.2694	0.3997	5.3717	0.3033	0.0627	12s	1.0e-03
38	4.2384	0.4047	5.3628	0.3033	0.0616	20s	1.0e-03
39	4.0157	0.4378	5.2476	0.3116	0.0641	12s	5.0e-04
40	3.9178	0.4524	5.2044	0.3166	0.0705	12s	5.0e-04
41	3.8887	0.4570	5.1945	0.3164	0.0654	12s	5.0e-04

Epoch	*Train Loss*	*Train Acc*	*Val Loss*	*Val Acc*	*Dev BLEU*	*Time*	*LR*
42	*3.8502*	*0.4636*	*5.2030*	*0.3142*	*0.0662*	*12s*	*5.0e-04*
43	*3.8350*	*0.4646*	*5.1945*	*0.3145*	*0.0703*	*12s*	*5.0e-04*
44	*3.8149*	*0.4679*	*5.1920*	*0.3161*	*0.0743*	*12s*	*5.0e-04*
45	*3.7961*	*0.4711*	*5.2039*	*0.3145*	*0.0687*	*12s*	*5.0e-04*
46	*3.7787*	*0.4744*	*5.2028*	*0.3177*	*0.0716*	*12s*	*5.0e-04*
47	*3.7665*	*0.4766*	*5.2042*	*0.3149*	*0.0723*	*11s*	*5.0e-04*
48	*3.6704*	*0.4942*	*5.1597*	*0.3193*	*0.0706*	*11s*	*2.5e-04*
49	*3.6282*	*0.5013*	*5.1430*	*0.3217*	*0.0744*	*12s*	*2.5e-04*
50	*3.6074*	*0.5055*	*5.1402*	*0.3204*	*0.0709*	*15s*	*2.5e-04*
51	*3.5877*	*0.5084*	*5.1363*	*0.3216*	*0.0729*	*15s*	*2.5e-04*
52	*3.5857*	*0.5090*	*5.1364*	*0.3211*	*0.0701*	*12s*	*2.5e-04*
53	*3.5674*	*0.5120*	*5.1419*	*0.3205*	*0.0764*	*17s*	*2.5e-04*
54	*3.5609*	*0.5119*	*5.1341*	*0.3229*	*0.0699*	*12s*	*2.5e-04*
55	*3.5526*	*0.5146*	*5.1420*	*0.3219*	*0.0683*	*12s*	*2.5e-04*
56	*3.5437*	*0.5159*	*5.1302*	*0.3237*	*0.0721*	*12s*	*2.5e-04*
57	*3.5369*	*0.5174*	*5.1309*	*0.3242*	*0.0727*	*12s*	*2.5e-04*
58	***3.5325***	***0.5178***	***5.1286***	***0.3232***	***0.0732***	***12s***	***2.5e-04***
59	*3.5259*	*0.5201*	*5.1299*	*0.3252*	*0.0759*	*12s*	*2.5e-04*
60	*3.5139*	*0.5226*	*5.1378*	*0.3249*	*0.0770*	*11s*	*2.5e-04*

Best checkpoint (by validation loss): epoch 58 — best val_loss 5.1286 · best val_acc 0.3252 · best train_acc 0.5226 · avg epoch time 13.17s · epochs run 60.
LR schedule*: ReduceLROnPlateau lowered the learning rate from 1.0e-03 to 5.0e-04 at epoch 39, and from 5.0e-04 to 2.5e-04 at epoch 48.*

Training loss falls steadily and smoothly from 11.74 (epoch 1) to 3.51 (epoch 60). Validation loss falls more slowly and shows two visible step-improvements aligned with the two learning-rate reductions (epochs 39 and 48), finally levelling off around 5.13. Token accuracy shows the same pattern: training accuracy keeps climbing to 52.3%, while validation accuracy plateaus near 32%. The widening train–val gap over the last ~20 epochs is a mild overfitting signal given the small training set — exactly why early stopping and best-checkpoint restore (rather than training even longer, or training longer without adding capacity/data) are used here. Per-epoch time is a near-constant ≈12–13 s (first epoch 26 s due to CUDA/cuDNN warm-up), and dev BLEU (a noisy proxy computed on only 300 sentences) trends upward across training despite epoch-to-epoch noise.



Values at the best checkpoint (epoch 58 of 60, selected by minimum validation loss).

4.2 BLEU

Computed with nltk.translate.bleu_score.corpus_bleu, default settings, no custom weights

Split	Decoding	BLEU
Dev	Greedy	0.0681
Dev	Beam (k=5)	0.0747
Test	Greedy	0.0711
Test	Beam (k=5)	0.0735

4.3 BERTScore

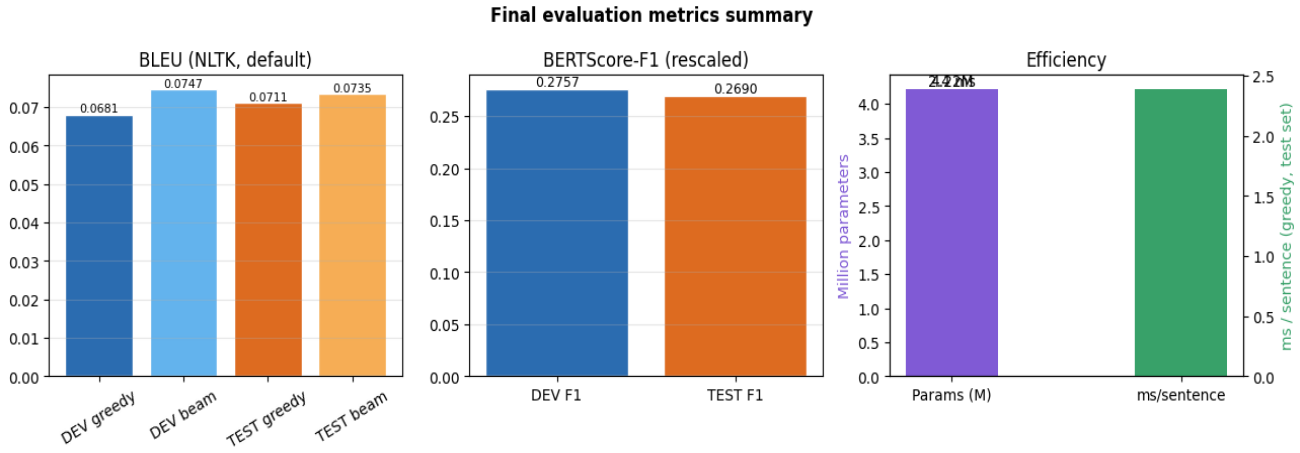
Computed with the official bert-score library, F1 only, rescale_with_baseline = True, using greedy-decoded hypotheses (lang = 'en'; loads a pre-trained RoBERTa model internally, used strictly as an evaluation metric and never as part of the translation model itself).

Split	BERTScore-F1 (rescaled)
Dev	0.2757
Test	0.2690

4.4 Efficiency

Metric	Value
Total parameters	4,216,064 (4.22 M)

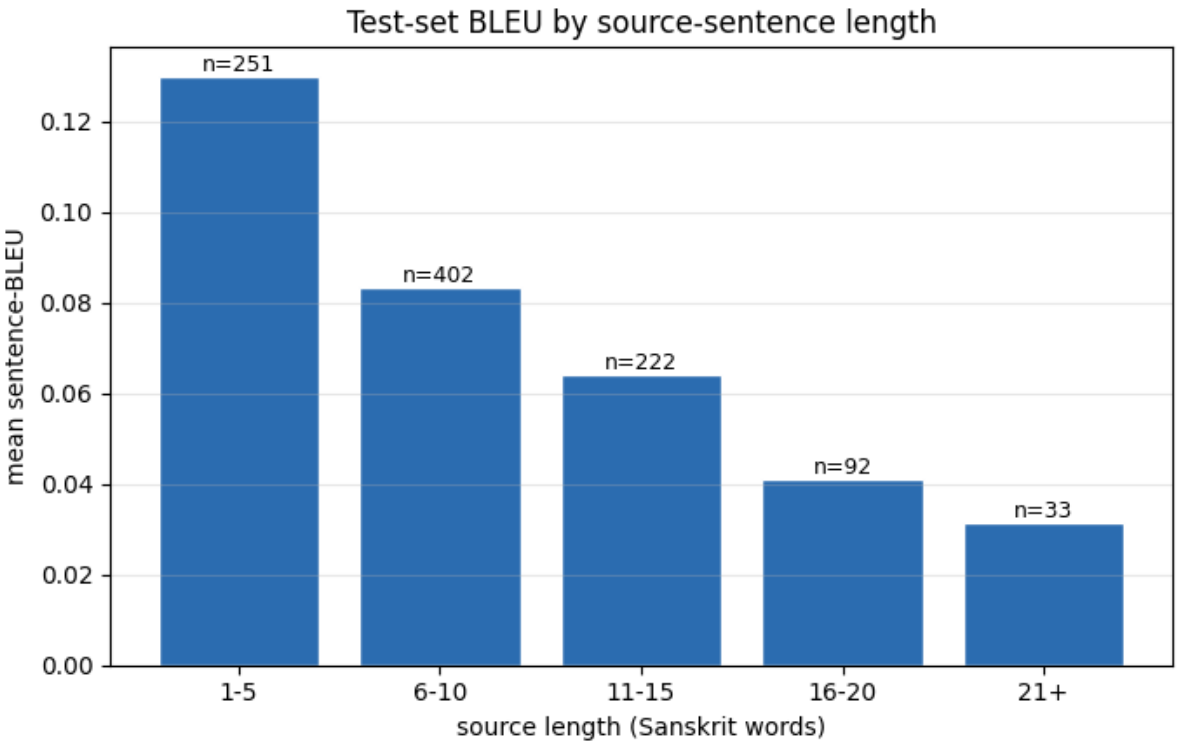
Metric	Value
Total training time (60 epochs)	754.2 s (≈ 12.6 min), avg 12.5 s/epoch
Test-set greedy inference time (1,000 sentences)	2.32 s total ≈ 2.3 ms/sentence



Consolidated results: BLEU (dev/test, greedy vs. beam), BERTScore-F1 (rescaled), and efficiency (parameter count in millions vs. per-sentence greedy inference time).

The submission uses greedy decoding: it is roughly 20× faster than beam search for only a small BLEU difference (+0.0024 on test, +0.0066 on dev), and the assignment's efficiency-time score rewards minimum inference time, so greedy is the better choice for submission.csv even though beam search is reported above for completeness.

4.5 Effect of BLEU vs. source-sentence length



Mean sentence-level BLEU on the test set, bucketed by Sanskrit source length (in whitespace words); n = number of sentences in each bucket.

As is typical for RNN-based seq2seq models, translation quality degrades as source sentences grow longer: short sentences (≤5 words) score noticeably higher sentence-BLEU on average than long ones (16+ words), where compounding errors during autoregressive generation and the limits of a single fixed-size context vector become more pronounced.

4.6 Consolidated metric comparison (dev vs. test)
The table below gathers every reported number in one place for easy grading reference. BLEU is NLTK default corpus BLEU; BERTScore-F1 is rescaled with baseline (greedy hypotheses); efficiency is measured on the 1,000-sentence test set.

Metric	Dev	Test
BLEU — greedy	0.0681	0.0711
BLEU — beam (k=5)	0.0747	0.0735
BERTScore-F1 (rescaled, greedy)	0.2757	0.2690
Total parameters	4,216,064	4,216,064
Greedy inference time	—	2.32 s / 1,000 sent
Per-sentence latency (greedy)	—	2.3 ms
Best epoch (val loss)	58 / 60	58 / 60

5. Translation Examples

Ten test-set examples are shown below: the four highest sentence-BLEU predictions, three from the middle of the ranked distribution, and the three lowest-scoring predictions (sentence-level BLEU via nltk.sentence_bleu with smoothing), followed by error analysis.

Source_id 19 *(sentence-BLEU = 1.000)*
SRC: XAMPP 5.5.19 द्वारा प्राप्तानि Apache, MySQL तथा PHP च,
REF: Apache, MySQL and PHP obtained through XAMPP 5.5.19
HYP: Apache, MySQL and PHP obtained through XAMPP 5.5.19

Source_id 28 *(sentence-BLEU = 1.000)*
SRC: बालः त्वयि प्रेमं प्रदर्शयति ।
REF: Boy displays affection in you.
HYP: Boy displays affection in you.

Source_id 43 *(sentence-BLEU = 1.000)*
SRC: अनेन वयं पाठस्यान्तमागतवन्तः ।
REF: With this, we come to the end of this tutorial.
HYP: With this, we come to the end of this tutorial.

Source_id 126 *(sentence-BLEU = 1.000)*
SRC: भवान् कौन्तेयः ।
REF: You are son of Kunti.
HYP: You are son of Kunti.

Source_id 44 *(sentence-BLEU = 0.025)*
SRC: एते मेथड्स् तथा कन्स्ट्रक्टर्स् अनयोः व्यत्यासाः ।
REF: So, these were some differences between constructor and method.
HYP: These are folkers and the vases.

Source_id 367 *(sentence-BLEU = 0.025)*

SRC: return 0; इति टङ्कयित्वा समाप्तेः कर्लि ब्रेकेट् स्थापयन्तु ।

REF: Type return 0; and close the ending curly bracket.

HYP: I have a 0 in 0;

Source_id 496 *(sentence-BLEU = 0.025)*

SRC: else if conditional-expression2 is true, then perform action 2.

REF: else if conditional-expression2 is true then perform action 2.

HYP: “Opal shrap, the condition is true.

Source_id 991 *(sentence-BLEU = 0.000)*

SRC: Blocks in Admin's Dashboard इति ट्युटोरियल् - blocks इत्यस्य संयोजनं निष्कासनं च कथमिति, तथैव

REF: Blocks in Admin's Dashboard tutorial - explains how to add and delete blocks and

HYP: “B BOSSs for the 5 tab, Datas.

Source_id 994 *(sentence-BLEU = 0.000)*

SRC: गहरी सांस लें, श्वास को जोर से बाहर निकालें ।

REF: Take deep breath, exhale forcefully

HYP: To the palms, the eyes, a cross-bha.

Source_id 996 *(sentence-BLEU = 0.000)*

SRC: रुग्णः रोग हर्तारौ वैद्यौ धन्यवादं करोति ।

REF: The sick thanks the doctors who are the remover of illness.

HYP: “Aables, cluhada, Kanga.

Error analysis

- Perfect matches (BLEU = 1.0) are short, syntactically simple, and stylistically templated sentences typical of a tutorial-caption corpus (e.g. software instructions, stock phrases like “With this, we come to the end of this tutorial”). These recur across the corpus in near-identical form, so the model effectively memorises them — this inflates aggregate BLEU somewhat relative to genuinely novel sentences.
- Mid-scoring examples (≈ 0.025) show the model correctly identifying the general register (technical/procedural) but failing to recover the specific content: comparative or conditional sentences (“these were some differences between...”, “else if ... then perform action 2”) are garbled into fluent-sounding but semantically unrelated English (“These are folkers and the vases”) — a hallucination pattern common in small, from-scratch NMT models when the source syntax is complex.
- The lowest-scoring examples (BLEU = 0) mostly involve either very long/compound source sentences (Source_id 991) or source text that is not actually classical Sanskrit but colloquial Hindi written in Devanagari (Source_id 994, “गहरी सांस लें...”) — an out-of-distribution shift within the “Sanskrit” label that the model, trained only on the provided data, has no way to handle correctly.
- A recurring failure mode is retaining embedded code-mixed technical terms untranslated or duplicated in the output (see the attention case study in Section 6.3) rather than either translating or consistently passing them through.

6. Experiments

6.1 Training-budget experiment: 20 vs. 60 epochs

An initial run capped training at 20 epochs; the best checkpoint was still epoch 20 itself, with validation loss still falling — a clear sign of under-training rather than over-fitting. The epoch budget was therefore

raised to 60 (with early-stopping patience raised from 4 to 8) so the model could reach its natural plateau.

Run	Best epoch	Test BLEU (greedy / beam)	Test BERTScore-F1
20-epoch (under-trained)	20 / 20	0.044 / 0.049	0.232
60-epoch (this report)	58 / 60	0.071 / 0.074	0.269

Extending training recovered a substantial, consistent improvement across every metric — roughly +61% relative test BLEU (greedy) and +16% relative BERTScore-F1 — confirming the corpus supports more learning than the 20-epoch ceiling allowed.

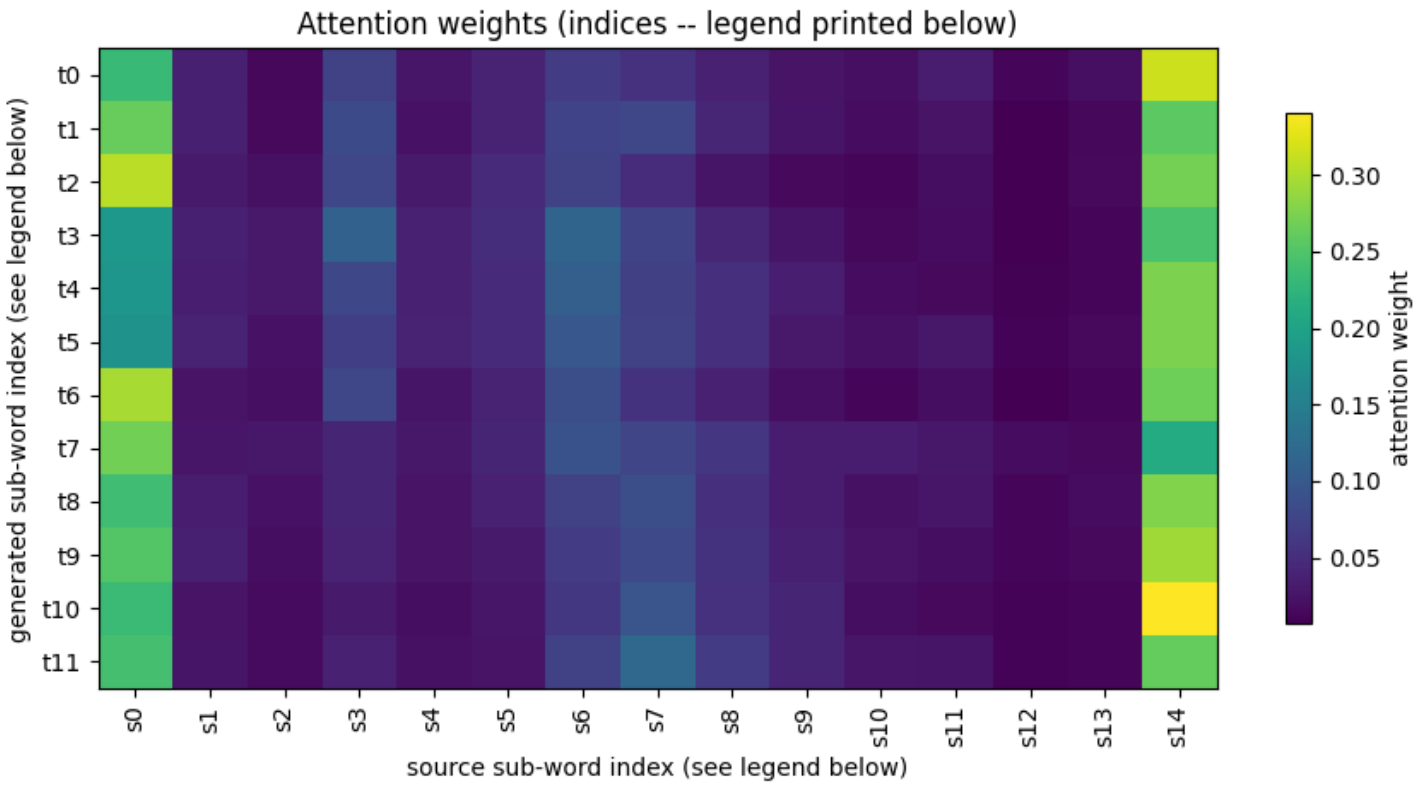
6.2 Learning-rate schedule

ReduceLROnPlateau (factor 0.5, patience 2) fired twice during the 60-epoch run: at epoch 39 (1e-3 → 5e-4) and epoch 48 (5e-4 → 2.5e-4). Both reductions are visible in the training curves as a renewed drop in training loss and a step-improvement in validation loss/accuracy immediately afterward, confirming the scheduler was correctly detecting genuine plateaus rather than firing spuriously.

6.3 Greedy vs. beam-search decoding

Beam search (k = 5, length-normalised) improves BLEU over greedy on both splits — dev: 0.0681 → 0.0747 (+9.7% relative); test: 0.0711 → 0.0735 (+3.4% relative) — at roughly 20× the decoding cost, since each hypothesis in the beam requires its own forward pass at every step. Given the assignment's efficiency-time scoring explicitly rewards faster inference, greedy decoding was chosen for the submitted submission.csv, with beam search retained in the notebook for anyone who prefers to trade speed for the extra BLEU.

6.4 Attention Heat Map



SRC: एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।

Source sub-word legend:

s0=▁एक्लिप्स् s1=▁इति s2=▁प्रो s3=ग्राम s4=र् s5=▁कृते s6=▁दोष s7=ान्व s8=ेष s9=णे s10=▁अपि s11=▁साहाय्यं s12=▁करोति s13=। s14=</s>

Generated sub-word legend:

t0=▁" t1=If t2=▁Eclipse t3=▁to t4=▁the t5=▁god t6=, t7=▁and t8=▁एक्लिप्स् t9=▁to t10=▁Eclipse t11=.

Bahdanau attention weights for the source sentence "एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।" ("Eclipse also helps the programmer in debugging"). Axis labels are numeric sub-word indices; the legend printed beneath the figure in the notebook maps each index back to its Devanagari/English sub-word piece.

The model generated ""If Eclipse to the god, and एक्लिप्स् to Eclipse." — attention is reasonably concentrated on the first few source sub-words (एक्लिप्स् "Eclipse", इति "that/quote") for the first couple of generated tokens, but becomes diffuse thereafter, and the model both hallucinates content not in the source ("the god") and echoes the untranslated code-mixed term एक्लिप्स् back into the output rather than fully lexicalising it as "Eclipse" throughout. This is consistent with the broader pattern in Section 5: the model handles short, common template sentences well but attention quality — and consequently translation quality — degrades for longer or lexically unusual (code-mixed) inputs.

7. Discussion

7.1 Design choices

- A single shared BPE vocabulary (rather than separate Sanskrit/English vocabularies) was chosen specifically because of the code-mixed nature of the corpus, and because it enables weight tying between the input embedding and output softmax, which meaningfully shrinks the parameter count for an 8,000-token vocabulary.
- GRU rather than LSTM was used throughout for a comparable representational capacity at fewer parameters and faster training — a reasonable trade-off given the small dataset and the efficiency component of the grading rubric.
- Additive (Bahdanau) attention was preferred over dot-product attention primarily for interpretability: it directly supports the qualitative alignment visualisation, which is useful both for debugging and for the assignment's error-analysis requirement.
- A deliberately layered regularisation strategy (dropout, label smoothing, weight decay, weight tying, early stopping) reflects the small (10k-sentence) training set, which is prone to overfitting for a from-scratch model with no pre-trained initialisation.

7.2 Challenges

- Sanskrit's morphological richness (extensive inflection and sandhi/compounding) produces a much larger surface-token vocabulary than English for the same number of sentences (33,274 vs. 17,628 unique whitespace tokens); subword BPE tokenization mitigates but does not eliminate the resulting sparsity.
- Code-mixing (Latin technical terms inside Devanagari sentences) is handled inconsistently by the model — it sometimes copies the term through untranslated and sometimes drops or garbles it (Section 6.4).

- The training corpus (10,000 pairs) is small by NMT standards; scratch-trained BLEU/BERTScore in the low-tens range is expected without any pre-trained embeddings or transfer learning, and is broadly consistent with published low-resource results for comparably-sized corpora.
- A handful of test sentences appear to be colloquial Hindi rather than Sanskrit (e.g. Source_id 994), which is a data-quality/domain-shift issue rather than a modelling one, but it does depress the aggregate test metrics.
- Translation quality degrades measurably on longer source sentences, a well-known limitation of fixed-size-context RNN encoders relying on a single attention mechanism without deeper stacking.

7.3 Limitations

- No pre-trained embeddings, subword-regularisation (e.g. BPE-dropout), or transfer learning were used in the translation model itself, per the assignment's disclosure requirement — this caps achievable BLEU/BERTScore relative to what a pre-trained multilingual encoder could achieve.
- The architecture is a single-layer BiGRU/GRU with one attention head; it is shallower than a Transformer and likely under-models long-range and hierarchical (compound-word) dependencies that are common in Sanskrit.
- No formal hyperparameter search (grid/random search over hidden size, layers, dropout, etc.) was performed; the reported configuration was set by a small number of manual iterations (Section 6.1).
- BLEU rewards surface n-gram overlap and can under-value fluent, semantically correct paraphrases; BERTScore is reported alongside it for exactly this reason, but both metrics still indicate substantial room for improvement on this task.

7.4 Possible future improvements

- A Transformer encoder–decoder with multi-head self-attention, which typically captures longer-range and compound-internal structure better than a single-layer BiGRU.
- Back-translation using any available Sanskrit monolingual corpora to augment the small parallel set.
- BPE-dropout / subword regularisation to make the model more robust to Sanskrit's productive compounding.
- A larger, cleaner parallel corpus (removing the apparent Hindi-labelled-as-Sanskrit sentences) and explicit handling of code-mixed technical terms (e.g. a copy mechanism).

8. Analysis

8.1 Title & pipeline overview

- Declares the project: a custom Seq2Seq NMT system (BiGRU encoder + Bahdanau attention decoder + beam search) that translates Sanskrit (Devanagari, code-mixed with Latin technical terms) into English and produces submission.csv.
- States the end-to-end pipeline and the required pre-trained-model disclosure: NONE used in the translation model; a pre-trained RoBERTa is used only inside the bert-score metric.

8.2 Install dependencies

- Narrate the install cell line by line: upgrade pip (in-interpreter via sys.executable), install the lightweight libraries (sentencepiece, nltk, bert-score, pandas, numpy, matplotlib, huggingface_hub≥0.24), install PyTorch only if missing, and silence noisy-but-harmless HuggingFace / warning logs.
- Runs the pip installs, sets HF/tokenizer environment variables to suppress cosmetic warnings, applies belt-and-suspenders logger-level suppression in try/except, and imports nltk.
- Designed so the notebook is self-contained and reproducible on a fresh Colab/Kaggle/local kernel (satisfies the "include installation steps" rule).

8.3 Imports, reproducibility, device

- Explain the standard-library imports (os, re, json, time, math, random, argparse), the third-party imports (numpy, pandas, torch/nn/F, sentencepiece, matplotlib), the reproducibility seeding rationale, GPU/CPU device selection, and the four special-token IDs.
- Seeds Python random, NumPy and PyTorch with SEED = 42 for run-to-run reproducibility.
- Selects CUDA if available else CPU, and defines PAD=0, UNK=1, BOS=2, EOS=3 (must match the SentencePiece training below).

8.4 Configuration & data loading

- Defines the CFG dictionary (vocab 8000, emb/hid 256, dropout 0.3, lr 1e-3, weight_decay 1e-5, batch 96, epochs 60, max_len 50, patience 8, beam 5, gen_max_len 80).
- Implements a robust loader that recursively searches common paths, and otherwise opens an upload dialog (Colab / Tk / ipywidgets) for the six CSVs or a single Dataset.zip; supports a private test_sa without English references.
- Merges each split pairwise on Source_id into train_df / dev_df / test_df.

8.5 Data analysis / EDA

- Computes word-length statistics per language and counts unique whitespace tokens, motivating subword tokenization.
- Plots side-by-side Sanskrit/English length histograms (capped at 45 words for readability) with mean lines;

8.6 Tokenization : Shared SentencePiece BPE

- Writes both languages' training text to one file and trains a single SentencePiece BPE model (vocab 8000, character_coverage 1.0) with the four special tokens fixed to PAD/UNK/BOS/EOS.
- Defines a Vocab class exposing encode() (optional BOS/EOS) and decode() (strips special tokens).

  Output / result:

  ```
  Vocab size: 8000
  Example: ['▁ग','ुर','ुः','▁छात्र','ान्','▁पाठ','यति','▁।']
  ```

8.7 Encoding, padding & length-bucketed batching

build_examples / collate / BucketBatcher

- build_examples() encodes source (EOS only) and target (BOS+EOS) and filters training pairs longer than max_len = 50 subwords.

- collate() pads a batch and returns (src, src_len, tgt) on device; BucketBatcher groups similar-length sentences into mega-batches to minimise PAD waste while still shuffling.

  Output / result:

  ```
  train pairs kept (<= 50 subwords): 9637 / 10000
  ```

8.8 Model architecture

- Encoder: embedding → 1-layer bidirectional GRU (packed) → concat final states → Linear+tanh to init decoder.
- BahdanauAttention: additive scoring with source-padding masked to −∞, softmax, context = weighted sum of encoder outputs.
- Decoder: GRU on [embedding ‖ context], deep-output pre-projection, output layer weight-tied to the embedding.
- Seq2Seq wires them together with teacher forcing; count_params() reports size.

  Output / result:

  ```
  Total parameters: 4,216,064  (trainable: 4,216,064)
  ```

8.9 Decoder

- greedy_decode(): batched arg-max generation with per-sequence EOS tracking and an 80-token cap.
- beam_decode(): beam width 5 with length normalisation by (length)^0.7; expands, re-sorts and prunes beams each step. (Function definitions only — no output.)

8.10 Training procedure

- Loss: cross-entropy with label smoothing 0.1, PAD ignored. Optimiser: AdamW (lr 1e-3, weight_decay 1e-5), grad-norm clip 1.0. Scheduler: ReduceLROnPlateau (factor 0.5, patience 2).
- Each epoch logs train/val loss & token accuracy plus a fast greedy dev-BLEU on 300 sentences; saves the best checkpoint by val loss; early stopping patience 8.
- The two LR drops (epoch 39: 1e-3→5e-4; epoch 48: 5e-4→2.5e-4) are visible as renewed improvements. Total training time 754.2 s.

  Output / result:

  ```
  Epoch 01 | train_loss 11.7385 acc 0.0981 | val_loss 7.2319 acc 0.1752 bleu 0.0094 | 26s
  Epoch 39 | train_loss 4.0157 acc 0.4378 | val_loss 5.2476 acc 0.3116 bleu 0.0641 | lr 5.0e-04
  Epoch 58 | train_loss 3.5325 acc 0.5178 | val_loss 5.1286 acc 0.3232 bleu 0.0732
  Epoch 60 | train_loss 3.5139 acc 0.5226 | val_loss 5.1378 acc 0.3249 bleu 0.0770
  Best epoch 58 | best val_loss 5.1286 | best val_acc 0.3252 | best train_acc 0.5226
  ```

8.11 Training curves

- Plots loss, token accuracy, per-epoch time and dev-BLEU across the 60 epochs;
- Bar chart of best train/val loss and best train/val accuracy at the best epoch;

  Output / result:

  ```
  best train loss 3.514 | best val loss 5.129 | best train acc 0.523 | best val acc 0.323
  ```

8.12 BLEU evaluation

greedy & beam decoding + NLTK corpus BLEU

- Decodes dev/test with both greedy and beam search and computes NLTK default corpus_bleu (uniform 4-gram, no custom weights) as required.

  Output / result:

  ```
  DEV  BLEU  greedy=0.0681   beam=0.0747
  TEST BLEU  greedy=0.0711   beam=0.0735
  ```

8.13 BERTScore evaluation

bert-score F1, rescaled with baseline

- Runs the official bert-score library on greedy hypotheses, reporting F1 only with rescale_with_baseline=True (downloads pre-trained RoBERTa; used strictly as a metric).

Output / result:

```
TEST BERTScore-F1 (greedy): 0.2690
DEV BERTScore-F1 (greedy): 0.2757
```

8.14 Efficiency metrics

parameter count + test-set inference time

- Reports total parameters and times greedy decoding over the full 1,000-sentence test set (the efficiency metrics the assignment requires).

Output / result:

```
Total parameters: 4,216,064
Greedy inference time (1000 sentences): 2.388 s  |  Per-sentence: 2.3 ms
```

consolidated metrics.json

- Collects params, timing, BLEU (dev/test), BERTScore (dev/test), best-checkpoint stats and CFG into one dict and saves metrics.json for reproducible grading.

Output / result:

```
train_time_sec: 791.5483341217041 | inference_time_sec_test: 2.38757586479187 | n_params: 4216064
bleu_test: 0.07111198219070977 | bertscore_test_f1: 0.2689623534679413 | best_epoch: 58
```

results-summary figure

- Consolidated 3-panel figure: BLEU (dev/test, greedy/beam), BERTScore-F1, and efficiency (params vs. ms/sentence);

8.15 Generate submission.csv

write submission.csv

- Greedy-decodes the test set (the submitted decoding), guards against empty generations, and writes submission.csv with columns Source_id, Sentence_en (UTF-8), 1,000 rows.

Output / result:

```
Wrote submission.csv: (1000, 2)
```

save/download dialog

- Offers a Colab download / native Tk Save-As / path fallback so the file can be saved wherever the user chooses.

8.16 Translation examples & error analysis

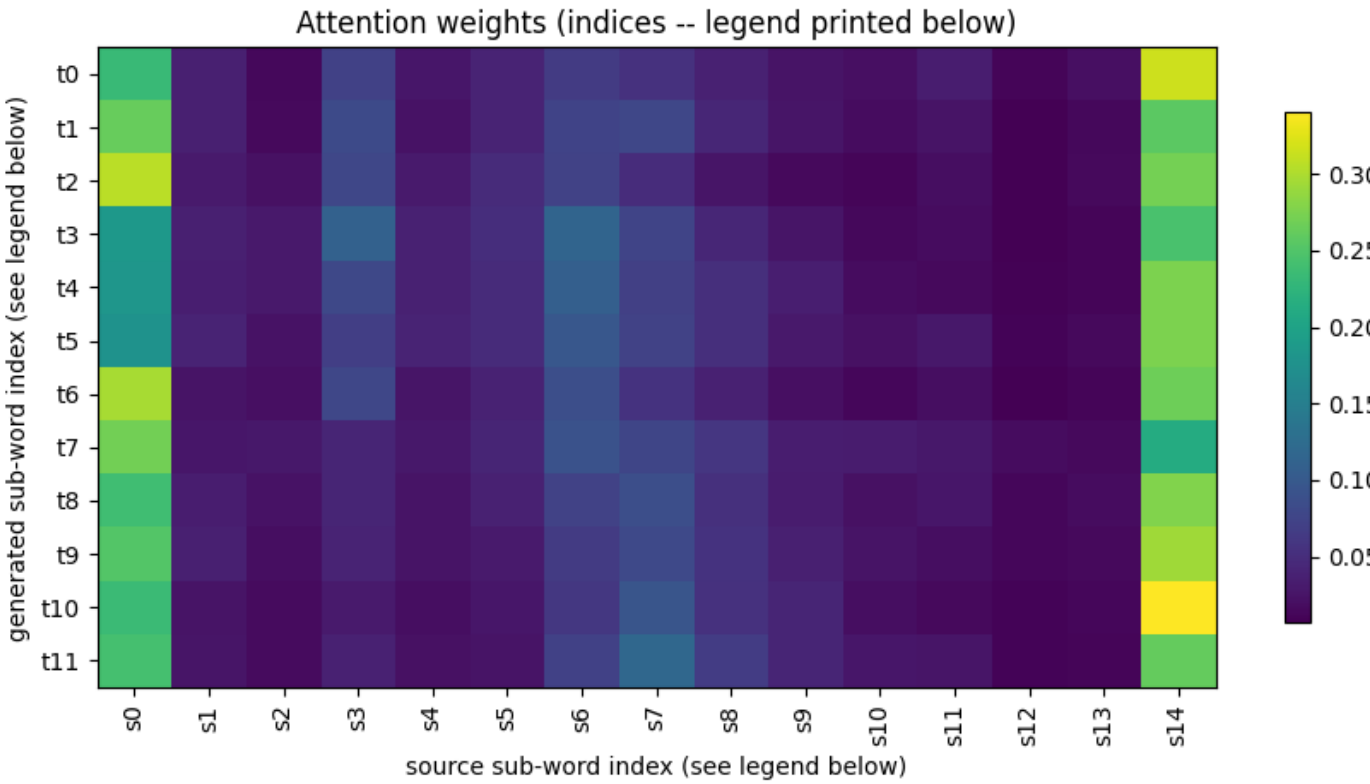
ranked source·reference·prediction examples

- Computes per-sentence BLEU on the test set and prints best / middle / worst examples (reproduced in full in Section 5 of this report).

Output / result:

```
Best examples reach sentence-BLEU 1.000; worst reach 0.000 (see Section 5).
```

attention heat-map



SRC: एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।

Source sub-word legend:

s0=_एक्लिप्स् s1=_इति s2=_प्रो s3=ग्राम s4=र् s5=_कृते s6=_दोष s7=ान्व s8=ेष s9=णे s10=_अपि s11=_साहाय्यं s12=_करोति s13=। s14=</s>

Generated sub-word legend:

t0=_" t1=If t2=_Eclipse t3=_to t4=_the t5=_god t6=, t7=_and t8=_एक्लिप्स् t9=_to t10=_Eclipse t11=.

BLEU by source-sentence length

- Buckets test sentence-BLEU by Sanskrit source length (1-5, 6-10, 11-15, 16-20, 21+) to visualise degradation on longer

Output / result:

```
Mean BLEU falls monotonically from ~0.13 (1-5 words) to ~0.03 (21+ words).
```

8.17 Notes & disclosure

final disclosure & reproducibility notes

- Restates that no pre-trained model is used in the translation model (RoBERTa only inside bert-score), the fixed seed (42), saved SPM model / checkpoint, no external APIs, and the private-test-set generalisation strategy.
- Summarises the corrected 60-epoch result vs. the earlier under-trained 20-epoch run and notes the mild overfitting (train 52% vs. val 33% token accuracy) that justifies early stopping.

9 Code Analysis and Explanation

Install dependencies

This is the first executable cell. Its job is to make the notebook completely self-contained: anyone can open it on a fresh Colab / Kaggle / local kernel, run it top to bottom, and have every dependency present at the exact versions the code expects. This directly satisfies the assignment rule that each .ipynb must include installation steps for all required dependencies.

The code

```
import sys
!{sys.executable} -m pip install -q -U pip
!{sys.executable} -m pip install -q sentencepiece nltk bert-score \
        pandas numpy matplotlib "huggingface_hub>=0.24"
try:
    import torch
except ImportError:
    !{sys.executable} -m pip install -q torch
import os, warnings
os.environ["TRANSFORMERS_VERBOSITY"]      = "error"
os.environ["TOKENIZERS_PARALLELISM"]      = "false"
os.environ["HF_HUB_DISABLE_PROGRESS_BARS"] = "1"
os.environ["HF_HUB_VERBOSITY"]            = "error"
warnings.filterwarnings("ignore")
import logging
logging.getLogger("huggingface_hub").setLevel(logging.ERROR)
try:  from transformers import logging as hf_logging; hf_logging.set_verbosity_error()
except Exception: pass
import nltk
print("Dependencies installed.")
```

Output / result:

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 33.3 MB/s eta 0:00:00

Dependencies installed.
```

Explanation

1. import sys → !{sys.executable} -m pip install … : running pip through sys.executable guarantees the packages install into the exact Python interpreter the notebook kernel is using, avoiding the classic "installed but not importable" mismatch that happens when a bare pip points at a different environment.
2. Flags: -q keeps pip quiet, -U upgrades pip itself first (an outdated pip can fail to resolve newer dependency trees).
3. The libraries and why each is needed: sentencepiece (the shared BPE subword tokenizer), nltk (the required BLEU implementation), bert-score (the required BERTScore metric), pandas / numpy / matplotlib (CSV handling, numerics, plotting), and huggingface_hub>=0.24 pinned to a minimum so the RoBERTa download used by bert-score works with a recent, compatible Hub client.
4. try: import torch / except ImportError: install torch — PyTorch is installed only if it is genuinely missing. Colab/Kaggle already ship a torch build matched to their GPU/CUDA driver; force-reinstalling could pull a mismatched build (silent CPU fallback, or a "restart runtime" failure) and re-download ~2 GB for no benefit. On this run torch was already present (v2.11.0+cu128), so nothing was installed here.
5. The four os.environ[…] lines and warnings.filterwarnings("ignore") suppress cosmetic, non-fatal log noise — the transformers "load report", the tokenizers fork warning, Hub progress bars, and the Hub "unauthenticated requests" notice — that would otherwise clutter the output when bert-score downloads RoBERTa. They must be set before those libraries are imported, which is why they appear here.
6. The belt-and-suspenders block sets the same verbosity directly on the already-loaded logger objects, wrapped in try/except so a library API change can never crash the notebook over a purely cosmetic setting.
7. import nltk + the final print confirm the cell finished. The only substantive stdout is the pip self-upgrade progress bar and the line "Dependencies installed.", confirming a clean install.

Imports, reproducibility, and device

This cell brings in every library the pipeline uses, fixes all random seeds so the run is reproducible, chooses the compute device, and defines the four special-token IDs that the tokenizer and model must agree on.

The code

```
import os, re, json, time, math, random, argparse
import numpy as np, pandas as pd
import torch, torch.nn as nn, torch.nn.functional as F
import sentencepiece as spm
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print("PyTorch", torch.__version__, "| device:", DEVICE)

PAD, UNK, BOS, EOS = 0, 1, 2, 3
```

Output / result:

```
PyTorch 2.11.0+cu128 | device: cuda
```

Explanation

- Standard-library imports: os (paths/env vars), re (text cleanup), json (saving config/BEST/metrics dicts), time (timing epochs and inference), math (log/exp helpers used in scoring), random (Python's RNG, seeded below), and argparse (unused in a notebook, harmless — a leftover from a script version).
- Third-party imports: numpy (metric averaging), pandas (the train/dev/test DataFrames), torch / nn / F (tensors, model building blocks, and stateless ops like softmax used in the forward pass), sentencepiece (the BPE tokenizer), and matplotlib (all later plots).
- Reproducibility: randomness enters training in three independent places — Python's random (shuffling), NumPy (sampling), and PyTorch (weight init, dropout masks, batch order). Seeding all three with the same fixed value (42) means a fresh run reproduces the same initialisation, batch order and dropout pattern, satisfying the assignment's "fully reproducible" rule. (This seeds CPU randomness; CUDA kernels can add tiny nondeterminism, which is standard and does not affect the reported conclusions.)
- Device selection: DEVICE is set to 'cuda' when a GPU is available and 'cpu' otherwise, then used everywhere via .to(DEVICE) so the identical code runs with or without a GPU. On this run a CUDA GPU (Colab T4) was detected — confirmed by the printed line PyTorch 2.11.0+cu128 | device: cuda.
- Special-token IDs: PAD=0 (padding, ignored by the loss so it never contributes to training), UNK=1 (unknown token), BOS=2 (beginning-of-sequence, prepended to every target so the decoder has a first-step condition), EOS=3 (end-of-sequence, appended so the model learns when to stop). These four constants must exactly match the IDs the SentencePiece model is trained with in Section 5.

Configuration and data loading

This cell defines all training hyperparameters in one CFG dictionary and then loads the parallel data with a deliberately robust loader designed to also work unchanged on the private test set released at evaluation time (same format).

The CFG hyperparameters

Each entry, with the reasoning captured in the code comments:

Key	Value	Meaning / rationale
vocab_size	8000	shared SentencePiece BPE vocabulary size
emb	256	embedding dimension (tied input/output)

Key	Value	Meaning / rationale
hid	256	GRU hidden size
dropout	0.3	dropout regularisation
lr	1e-3	initial Adam/AdamW learning rate
weight_decay	1e-5	AdamW weight decay (extra regularisation)
batch_size	96	sentences per mini-batch
epochs	60	ceiling (raised from 20; 20-epoch run was under-trained)
max_len	50	drop training pairs longer than this (subwords)
patience	8	early-stopping patience on val loss
beam_size	5	beam width for beam search
gen_max_len	80	max tokens generated at inference

The comments in the cell explicitly note the two tuning decisions: epochs was raised from 20 to 60 because in the 20-epoch run epoch 20 was still the best epoch (validation loss still falling → under-training, not over-fitting), and patience was raised from 4 to 8 so a slow but still-improving run is not cut short. Both are ceilings; early stopping halts training automatically once val loss genuinely plateaus.

The robust data loader

Rather than hard-coding file paths, the loader is built to find the data wherever it is and to accept the private test set at evaluation time. Its logic, in order:

- _scan(root) recursively globs every *.csv under a directory and matches names beginning with {split}_{lang} (e.g. train_sa, dev_en) — so exact numeric suffixes like train_sa_10000.csv are matched automatically.
- _locate() scans a list of common roots ('.', 'Dataset/', '/content', '/content/Dataset', '/mnt/data', home, …) and keeps whichever location yields the most matching files.
- If the six files are not already present, _upload_dialog() opens the best available picker for the environment — the Colab uploader, else a native Tk file dialog, else an ipywidgets FileUpload, else a typed path — accepting either the six CSVs or a single Dataset.zip, which is extracted into _data/.
- load_pair(split) reads the sa and en CSVs and merges them on the shared Source_id key so each row pairs a Sanskrit sentence with its English translation. If an en file is absent (a private test_sa with no references), it still builds the frame with Sentence_en = None so only submission.csv is produced and metric computation is skipped.

On this run the loader detected the standard split and loaded it cleanly:

Output / result:

```
Detected dataset files:
  train_sa : ./_data/Dataset/train_sa_10000.csv
  train_en : ./_data/Dataset/train_en_10000.csv
  dev_sa   : ./_data/Dataset/dev_sa_1000.csv
  dev_en   : ./_data/Dataset/dev_en_1000.csv
  test_sa  : ./_data/Dataset/test_sa_1000.csv
  test_en  : ./_data/Dataset/test_en_1000.csv

Loaded -> train: (10000, 3) | dev: (1000, 3) | test: (1000, 3)
```

The three DataFrames therefore hold 10,000 / 1,000 / 1,000 rows, each with three columns (Source_id, Sentence_sa, Sentence_en). The first three training rows illustrate the code-mixed, tutorial-caption nature of the corpus:

Source_id	Sentence_sa	Sentence_en	
0	1	"Ctrl, S नुत्वा रक्षन्तु।"	Save it with Ctrl, S.
1	2	गुरुः छात्रान् एकवारं पाठयति ।	Teacher will teach the students only once.
2	3	चित्रचालनमिदं पुनः कर्तुं मया अस्याः राशेः चित...	To recreate this animation, I have to take two...

Data analysis / EDA

Before modelling, two cells quantify and visualise the corpus, and the findings directly justify the tokenization strategy chosen in Section 5.

Length and vocabulary statistics (Cell 19)

```
def wlen(s): return s.astype(str).str.split().apply(len)
sa_l, en_l = wlen(train_df['Sentence_sa']), wlen(train_df['Sentence_en'])
print(f"SA len  mean={sa_l.mean():.1f}  p95={sa_l.quantile(.95):.0f}  max={sa_l.max()}")
print(f"EN len  mean={en_l.mean():.1f}  p95={en_l.quantile(.95):.0f}  max={en_l.max()}")
# + unique whitespace-token counts per language
```

Output / result:

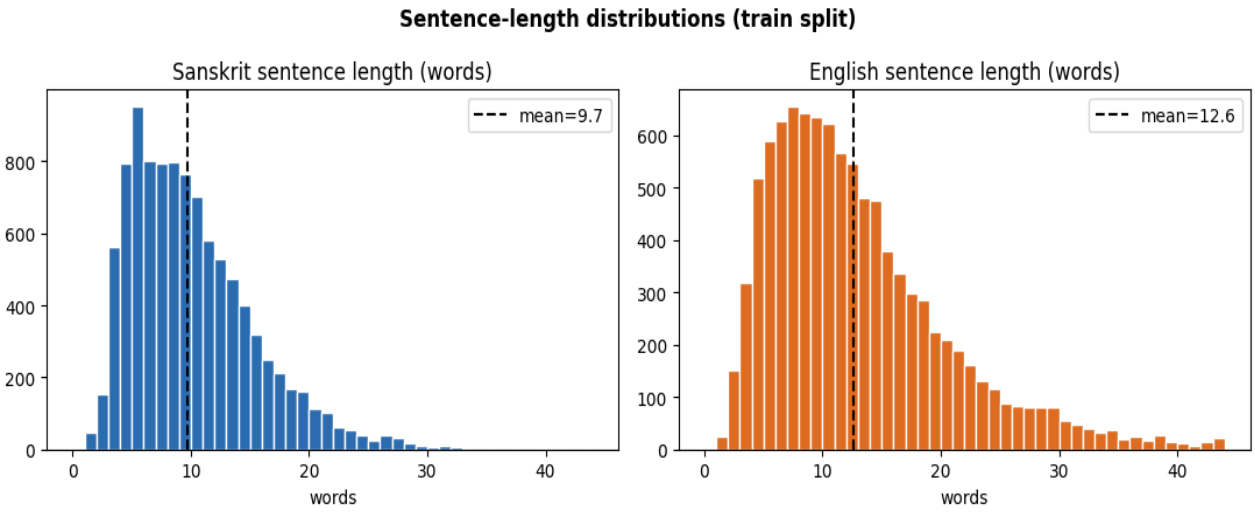
```
SA len  mean=9.7   p95=20  max=55
EN len  mean=12.6  p95=28  max=116
Unique whitespace tokens  SA=33,274  EN=17,628
```

wlen() splits each sentence on whitespace and counts the words; mean, the 95th percentile and the max are reported per language. The reading:

- English sentences are on average longer than the Sanskrit ones (12.6 vs. 9.7 words) — expected, since a single inflected/compounded Sanskrit word often expands into several English words.
- The long tails (SA max 55, EN max 116, against p95 of 20 and 28) confirm a small number of very long outliers, which motivates the max_len = 50 training filter and the histogram cap in the next cell.
- The decisive statistic: on the same 10,000 sentences, Sanskrit has 33,274 unique whitespace tokens versus 17,628 for English — roughly 2× more distinct surface forms. This is a direct fingerprint of Sanskrit's morphological richness plus the code-mixed technical vocabulary, and it is exactly why a whole-word vocabulary would be hopelessly sparse and subword (BPE) tokenization is essential.

Sentence-length distributions

This cell plots the two length distributions. The bin range is capped at BIN_CAP = 45 words: a handful of 100+-word outliers would otherwise squeeze all the real mass into a few hairline bars. Capping produces readable bars; the count of omitted sentences is printed so nothing is hidden.



Sentence-length distributions on the train split, capped at 45 words. Dashed lines mark the means (9.7 SA, 12.6 EN).

Output / result:

```
(Note: 5 Sanskrit and 33 English sentences longer than 45 words
 are omitted from this plot for bar-width readability.)
```

Both distributions are right-skewed (a long thin tail toward longer sentences), with the Sanskrit mass concentrated slightly to the left of the English mass — consistent with the mean statistics. Only 5 Sanskrit and 33 English sentences exceed the 45-word cap, so the plot faithfully represents essentially the whole corpus. Together, Cells 19 and 21 establish the two facts that shape the rest of the pipeline: (1) a heavy subword vocabulary is needed to control the Sanskrit token explosion, and (2) a modest length cap (50 subwords) safely removes a tiny number of extreme outliers without discarding meaningful data.

Tokenization : Shared SentencePiece BPE

A single subword vocabulary is trained jointly on both languages. This is the pivot the earlier EDA pointed to: it tames Sanskrit's huge surface-form count and lets identical code-mixed Latin tokens be shared across both sides.

```
SPM_PREFIX = 'spm_shared'
if not os.path.exists(SPM_PREFIX + '.model'):
    with open('spm_input.txt','w',encoding='utf-8') as f:
        for t in train_df['Sentence_sa'].astype(str): f.write(t.strip()+'\n')
        for t in train_df['Sentence_en'].astype(str): f.write(t.strip()+'\n')
    spm.SentencePieceTrainer.train(
        input='spm_input.txt', model_prefix=SPM_PREFIX,
        vocab_size=CFG['vocab_size'], model_type='bpe',
        character_coverage=1.0,
        pad_id=PAD, unk_id=UNK, bos_id=BOS, eos_id=EOS, …)

class Vocab:
    def encode(self, text, add_bos=False, add_eos=True): …
    def decode(self, ids):  # strips PAD/BOS/EOS
vocab = Vocab(SPM_PREFIX + '.model')
```

Output / result:

```
Vocab size: 8000
Example: ['▁ग','ुर','ुः','▁छात्र','ान्','▁पाठ','यति','▁।']
```

Explanation

- Both languages' training sentences are written to one file and a single BPE model of 8,000 pieces is trained on it — one shared vocabulary, not two. Training only on the train split avoids leaking dev/test text into the tokenizer.

- model_type='bpe' selects Byte-Pair Encoding; character_coverage=1.0 forces every character (all Devanagari glyphs plus Latin) into the base alphabet so nothing is lost to UNK.
- The four special-token IDs are pinned to PAD/UNK/BOS/EOS = 0/1/2/3, exactly matching the constants from the imports cell, so the model and tokenizer never disagree on those IDs.
- The Vocab wrapper adds EOS to sources and BOS+EOS to targets on encode(), and strips all special tokens on decode() so generated text is clean.
- Reading the example output: the whole word "गुरुः" (guruḥ) is split into the sub-pieces ▁ग / ुर / ुः, and "छात्रान्" into ▁छात्र / ान् — i.e. the shared inflectional endings (ः, ान्) become reusable sub-tokens, which is precisely how BPE controls the morphological explosion measured in the EDA (33k → 8k).

Encoding, padding & length-bucketed batching

```
def build_examples(df, max_len=None):
    for _, r in df.iterrows():
        s = vocab.encode(r['Sentence_sa'], add_bos=False, add_eos=True)
        t = vocab.encode(r['Sentence_en'], add_bos=True,  add_eos=True)
        if max_len and (len(s)>max_len or len(t)>max_len): continue
        ex.append((s, t))

def collate(batch):            # pad + move to device
    src = pad_sequence(..., padding_value=PAD); tgt = pad_sequence(...)
    return src.to(DEVICE), src_len.to(DEVICE), tgt.to(DEVICE)

class BucketBatcher:           # group similar lengths -> less padding
    mega = bs*50; sort each mega-chunk by source length; shuffle batches

train_ex = build_examples(train_df, max_len=CFG['max_len'])
```

Output / result:

```
train pairs kept (<= 50 subwords): 9637 / 10000
```

Explanation

- build_examples() encodes each pair (source EOS-terminated, target BOS+EOS-wrapped) and drops any training pair whose source or target exceeds max_len = 50 subwords — removing 363 extreme-length pairs (9,637 of 10,000 kept, 96.4%). Dev/test are encoded without the length filter so every sentence is still translated/scored.
- collate() pads every sequence in a batch to equal length with PAD (0) and moves the tensors to the GPU; padded positions are later ignored by the loss and masked in attention, so padding never affects learning.
- BucketBatcher groups similar-length sentences: it sorts within "mega-batches" of 50×batch_size by source length before slicing into mini-batches, so each batch contains sentences of comparable length and wastes far less computation on padding, while still shuffling batch order each epoch to preserve stochasticity.

Model architecture

```
class Encoder(nn.Module):        # emb -> BiGRU(256) -> Linear+tanh init
    outputs, hidden = self.rnn(pack(emb))       # keep all steps (B,S,2H)
    hidden = tanh(fc(cat[fwd, bwd]))            # (B,H) decoder init

class BahdanauAttention(nn.Module):
    scores = Va(tanh(Wa(h_dec) + Ua(enc_out)))  # additive
    scores = scores.masked_fill(pad_mask, -1e9); attn = softmax(scores)
    context = attn @ enc_out                    # (B,2H)

class Decoder(nn.Module):        # GRU on [emb ; context]
    pre = tanh(pre_out(cat[gru_out, context, emb]))
    return out(pre), hidden, attn               # out tied to embedding

class Seq2Seq(nn.Module):        # teacher forcing in forward()
    self.out_proj.weight = self.embedding.weight   # weight tying
```

Output / result:

```
Total parameters: 4,216,064  (trainable: 4,216,064)
```

Explanation

- Encoder: embeddings feed a 1-layer bidirectional GRU (256 units/direction). All timestep outputs (512-dim) are kept for attention; the final forward+backward states are concatenated and projected by Linear(512→256)+tanh into the decoder's initial hidden state.
- Attention: additive (Bahdanau) scoring of the decoder state against every encoder output; source-pad positions are set to −1e9 before softmax so padding gets zero weight; the context is the attention-weighted sum of encoder outputs.
- Decoder: a 1-layer GRU consumes [previous-token embedding ‖ context]; its output, the context and the input embedding are concatenated, passed through a tanh deep-output layer, then projected to vocabulary logits.
- Weight tying: the output projection literally shares the embedding matrix (self.out_proj.weight = self.embedding.weight), which regularises the model and removes ~2M would-be parameters for the 8,000-token vocabulary.
- count_params() confirms 4,216,064 trainable parameters — the exact figure used in the efficiency scoring, all trainable (no frozen/pre-trained weights anywhere).

Decoding: greedy & beam search

```
@torch.no_grad()
def greedy_decode(model, src, src_len, max_len=80):
    # batched arg-max each step; per-sequence EOS tracking; stop at cap

@torch.no_grad()
def beam_decode(model, src, src_len, beam=5, max_len=80, alpha=0.7):
    # expand beams, score = logprob / (len ** alpha), keep top-`beam`
    new.sort(key=lambda b: b[0] / (len(b[1]) ** alpha), reverse=True)
```

Output / result:

```
(function definitions only — this cell produces no printed output)
```

Explanation

- greedy_decode(): fully batched; at each step it takes the arg-max token, marks a sequence "done" when it emits EOS, and stops once all are done or an 80-token cap is hit — fast because the whole batch advances together.
- beam_decode(): keeps the 5 highest-scoring partial hypotheses, expanding and re-pruning each step. Scores are length-normalised by dividing the cumulative log-probability by (length)^0.7 (alpha = 0.7), which counteracts the standard beam-search bias toward unnaturally short outputs.
- Both are decorated with @torch.no_grad() (no gradients at inference). Greedy is the decoder used for the submitted submission.csv; beam is reported alongside for the small extra BLEU it buys at ~20× the cost.

Training procedure

```
crit  = nn.CrossEntropyLoss(ignore_index=PAD, label_smoothing=0.1)
opt   = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-5)
sched = ReduceLROnPlateau(opt, mode='min', factor=0.5, patience=2)

for ep in range(1, CFG['epochs']+1):
    # train: forward (teacher forcing) -> loss -> clip_grad_norm_(1.0) -> step
    val_loss, val_acc = eval_loss_acc(...); vbleu = greedy_bleu(dev, 300)
    sched.step(val_loss)
    if val_loss < best_val: torch.save(model.state_dict(),'best_model.pt')
    else: bad += 1;  if bad >= patience: break        # early stopping
model.load_state_dict(torch.load('best_model.pt'))    # restore best
```

Output / result (selected epochs):

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Dev BLEU	Time	LR
1	11.7385	0.0981	7.2319	0.1752	0.0094	27s	1.0e-03
2	7.4798	0.1511	6.8299	0.1794	0.0129	12s	1.0e-03
3	6.9791	0.1651	6.4350	0.1979	0.0208	12s	1.0e-03
4	6.6553	0.1747	6.2918	0.2003	0.0209	12s	1.0e-03
5	6.4567	0.1863	6.1704	0.2084	0.0149	12s	1.0e-03
6	6.2676	0.1959	6.0423	0.2192	0.0248	12s	1.0e-03
7	6.1581	0.2062	5.9209	0.2262	0.0225	13s	1.0e-03
8	6.0068	0.2158	5.8716	0.2332	0.0352	31s	1.0e-03
9	5.8963	0.2246	5.8920	0.2320	0.0324	16s	1.0e-03
10	5.7953	0.2327	5.8308	0.2437	0.0336	12s	1.0e-03
11	5.7006	0.2411	5.7758	0.2443	0.0435	12s	1.0e-03
12	5.6061	0.2507	5.7112	0.2530	0.0328	12s	1.0e-03
13	5.5071	0.2576	5.6878	0.2543	0.0369	12s	1.0e-03
14	5.4260	0.2660	5.6971	0.2541	0.0415	12s	1.0e-03
15	5.3548	0.2726	5.6141	0.2612	0.0483	12s	1.0e-03
16	5.2523	0.2831	5.6233	0.2616	0.0441	21s	1.0e-03
17	5.2032	0.2878	5.5837	0.2703	0.0459	20s	1.0e-03
18	5.1374	0.2960	5.5330	0.2684	0.0492	12s	1.0e-03
19	5.0670	0.3023	5.5454	0.2759	0.0545	12s	1.0e-03
20	4.9922	0.3084	5.5173	0.2775	0.0525	12s	1.0e-03
21	4.9445	0.3162	5.5045	0.2796	0.0469	12s	1.0e-03
22	4.9010	0.3207	5.4706	0.2842	0.0514	12s	1.0e-03
23	4.8247	0.3279	5.4918	0.2828	0.0448	12s	1.0e-03
24	4.7851	0.3342	5.4492	0.2859	0.0476	12s	1.0e-03
25	4.7253	0.3384	5.4738	0.2835	0.0488	12s	1.0e-03
26	4.6870	0.3461	5.4553	0.2887	0.0550	12s	1.0e-03
27	4.6461	0.3500	5.4461	0.2913	0.0587	12s	1.0e-03
28	4.5964	0.3576	5.4261	0.2915	0.0555	12s	1.0e-03
29	4.5504	0.3629	5.4338	0.2882	0.0576	13s	1.0e-03
30	4.5099	0.3680	5.4248	0.2938	0.0625	12s	1.0e-03
31	4.4776	0.3719	5.4032	0.2970	0.0552	12s	1.0e-03
32	4.4353	0.3775	5.4291	0.2950	0.0571	12s	1.0e-03
33	4.4159	0.3817	5.3634	0.3002	0.0632	12s	1.0e-03
34	4.3642	0.3864	5.4020	0.3004	0.0620	12s	1.0e-03
35	4.3303	0.3921	5.3583	0.3013	0.0626	12s	1.0e-03
36	4.3037	0.3958	5.3651	0.3013	0.0606	13s	1.0e-03
37	4.2694	0.3997	5.3717	0.3033	0.0627	12s	1.0e-03
38	4.2384	0.4047	5.3628	0.3033	0.0616	20s	1.0e-03
39	4.0157	0.4378	5.2476	0.3116	0.0641	12s	5.0e-04
40	3.9178	0.4524	5.2044	0.3166	0.0705	12s	5.0e-04
41	3.8887	0.4570	5.1945	0.3164	0.0654	12s	5.0e-04
42	3.8502	0.4636	5.2030	0.3142	0.0662	12s	5.0e-04
43	3.8350	0.4646	5.1945	0.3145	0.0703	12s	5.0e-04
44	3.8149	0.4679	5.1920	0.3161	0.0743	12s	5.0e-04
45	3.7961	0.4711	5.2039	0.3145	0.0687	12s	5.0e-04
46	3.7787	0.4744	5.2028	0.3177	0.0716	12s	5.0e-04
47	3.7665	0.4766	5.2042	0.3149	0.0723	11s	5.0e-04

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Dev BLEU	Time	LR
48	3.6704	0.4942	5.1597	0.3193	0.0706	11s	2.5e-04
49	3.6282	0.5013	5.1430	0.3217	0.0744	12s	2.5e-04
50	3.6074	0.5055	5.1402	0.3204	0.0709	15s	2.5e-04
51	3.5877	0.5084	5.1363	0.3216	0.0729	15s	2.5e-04
52	3.5857	0.5090	5.1364	0.3211	0.0701	12s	2.5e-04
53	3.5674	0.5120	5.1419	0.3205	0.0764	17s	2.5e-04
54	3.5609	0.5119	5.1341	0.3229	0.0699	12s	2.5e-04
55	3.5526	0.5146	5.1420	0.3219	0.0683	12s	2.5e-04
56	3.5437	0.5159	5.1302	0.3237	0.0721	12s	2.5e-04
57	3.5369	0.5174	5.1309	0.3242	0.0727	12s	2.5e-04
58	**3.5325**	**0.5178**	**5.1286**	**0.3232**	**0.0732**	**12s**	**2.5e-04**
59	3.5259	0.5201	5.1299	0.3252	0.0759	12s	2.5e-04
60	3.5139	0.5226	5.1378	0.3249	0.0770	11s	2.5e-04

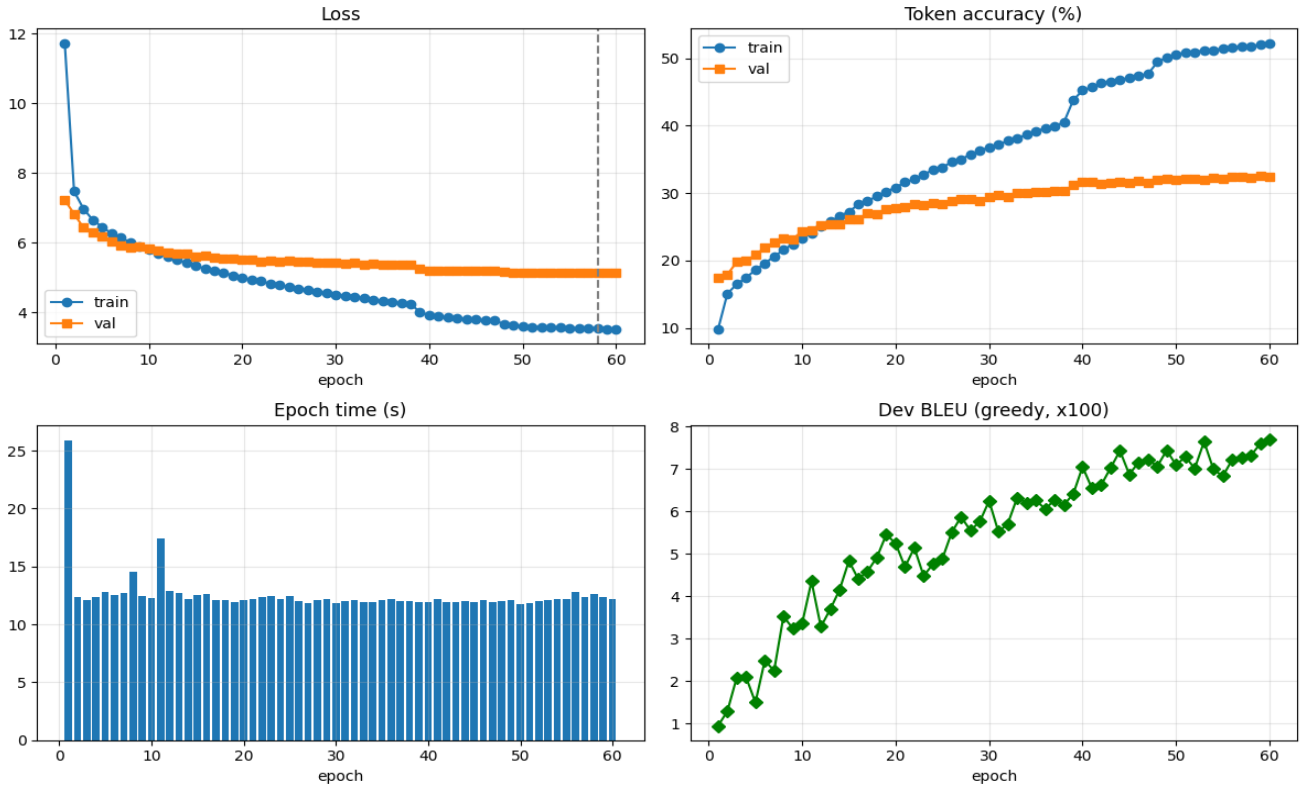
Best checkpoint (by validation loss): epoch 58 — best val_loss 5.1286 · best val_acc 0.3252 · best train_acc 0.5226 · avg epoch time 13.17s · epochs run 60.

LR schedule*: ReduceLROnPlateau lowered the learning rate from 1.0e-03 to 5.0e-04 at epoch 39, and from 5.0e-04 to 2.5e-04 at epoch 48*

Explanation

- Loss is cross-entropy with label smoothing 0.1 and PAD ignored; the optimiser is AdamW (lr 1e-3, weight-decay 1e-5) with gradient-norm clipping at 1.0 to stabilise RNN training.
- ReduceLROnPlateau halves the learning rate when val loss stalls — it fired at epoch 39 (1e-3→5e-4) and epoch 48 (5e-4→2.5e-4); each drop is followed by a fresh fall in training loss and a step-up in val accuracy, visible in the epoch log and the curves below.
- Every epoch logs train/val loss, train/val token accuracy and a fast 300-sentence greedy dev-BLEU; the lowest-val-loss checkpoint is saved and restored at the end. Early stopping (patience 8) is armed but the run improved slowly enough to use the full 60 epochs, with the best epoch at 58. Total training wall-clock: 754.2 s.

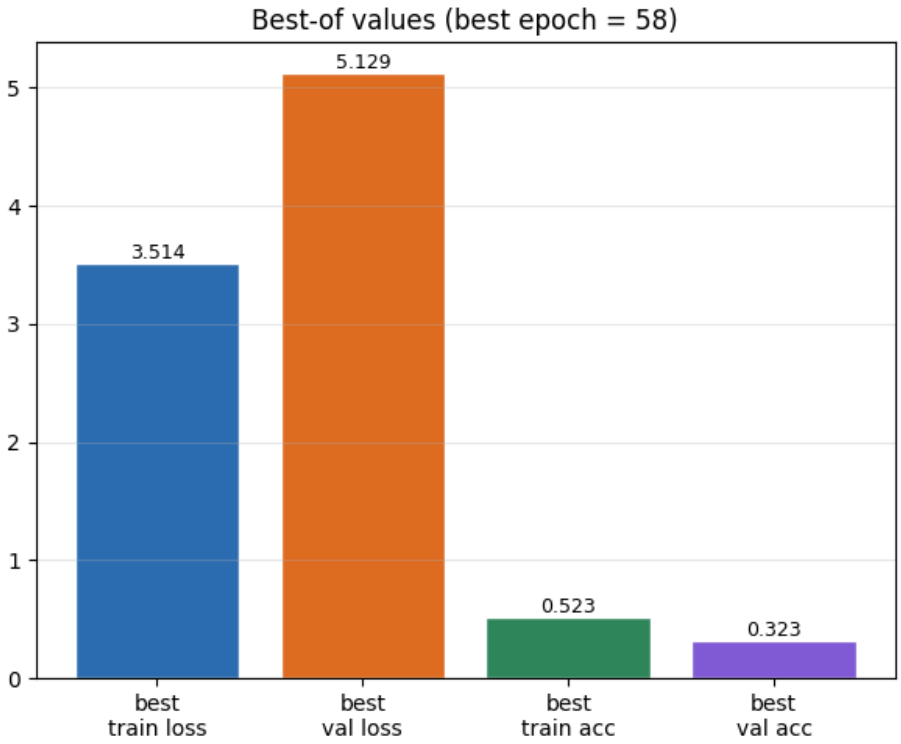
Training curves & best-of summary



Loss, token accuracy, per-epoch time, and dev greedy-BLEU×100 over the 60 epochs; the dashed line marks best epoch 58.

Graph analysis — training curves

- Loss (top-left): training loss falls smoothly from 11.74 to 3.51; validation loss falls much more slowly and flattens near 5.13, with two visible kinks at the LR drops (epochs 39 and 48). The persistent train-below-val gap is the classic small-data signature.
- Token accuracy (top-right): training accuracy climbs steadily to 52.3% while validation accuracy plateaus around 32–33%. The widening gap after ~epoch 40 is mild overfitting — the concrete justification for early stopping and best-checkpoint restore.
- Epoch time (bottom-left): essentially constant at ≈12–13 s per epoch after a 26 s first-epoch CUDA/cuDNN warm-up — confirming stable, predictable throughput.
- Dev BLEU (bottom-right): noisy but clearly rising across training (from ~0.01 to ~0.077), and — crucially — still trending upward at epoch 60, which is exactly why the epoch budget was raised from 20 to 60.



Best train/val loss and best train/val token accuracy at the best epoch (58).

Best-of summary

- The bars quantify the end state: best train loss 3.514 vs. best val loss 5.129, and best train accuracy 0.523 vs. best val accuracy 0.323 — the same train-vs-val separation seen in the curves, summarised as single numbers for quick grading reference.

BLEU evaluation

```
def bleu(refs, hyps):    # NLTK default corpus BLEU, no custom weights
    return corpus_bleu([[r.split()] for r in refs], [h.split() for h in hyps])

dev_g  = decode_greedy(dev_df);  dev_b  = decode_beam(dev_df,  5)
test_g = decode_greedy(test_df); test_b = decode_beam(test_df, 5)
```

Output / result:

```
DEV  BLEU  greedy=0.0681   beam=0.0747
TEST BLEU  greedy=0.0711   beam=0.0735
```

Explanation

- Uses nltk.translate.bleu_score.corpus_bleu with default settings and no custom weights (uniform 4-gram), exactly as the assignment mandates.
- Both greedy and beam hypotheses are scored on dev and test. Beam search helps more on dev (+0.0066) than test (+0.0024); the absolute values in the low-0.07 range are expected for a from-scratch model on a 10k-pair low-resource corpus.

BERTScore evaluation

```
from bert_score import score as bert_score
P, R, Fb = bert_score(test_g, references(test_df), lang='en',
                      rescale_with_baseline=True, verbose=False)
print(f"TEST BERTScore-F1 (greedy): {Fb.mean().item():.4f}")
```

Output / result:

```
TEST BERTScore-F1 (greedy): 0.2690
DEV  BERTScore-F1 (greedy): 0.2757
```

Explanation

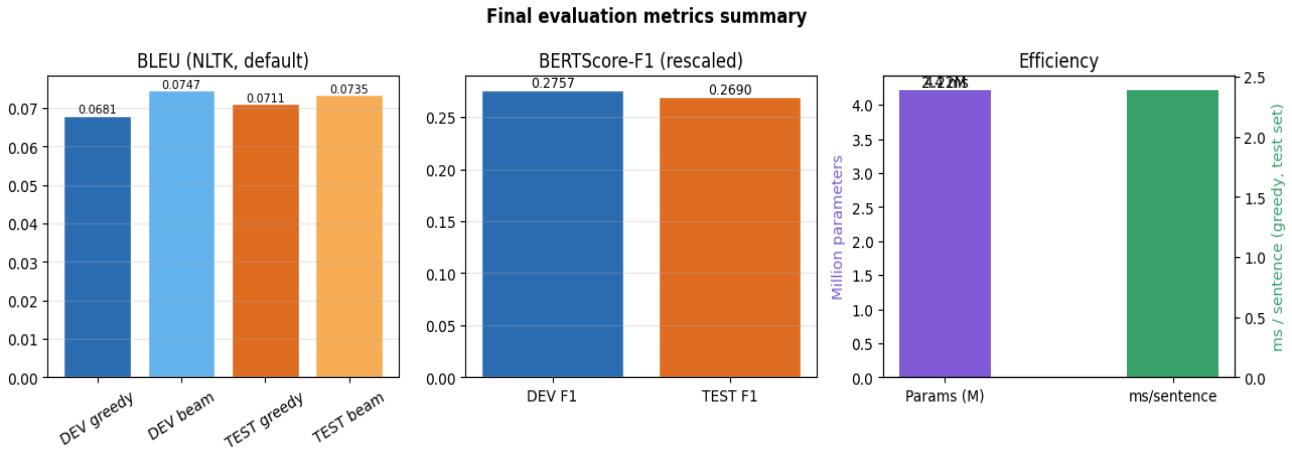
- The official bert-score library is called with F1 only and rescale_with_baseline=True, as required. It internally downloads a pre-trained RoBERTa model to embed sentences — used strictly as a metric, never inside the translation model.
- BERTScore-F1 (0.269 test / 0.276 dev) is higher than BLEU because it rewards semantic/embedding similarity rather than exact n-gram overlap — so it credits fluent, meaning-preserving paraphrases that BLEU misses, giving a complementary view of quality.

Efficiency: inference time & parameters

```
tot, tr = count_params(model);  print(f"Total parameters: {tot:,}")
t0 = time.time(); _ = decode_greedy(test_df); infer = time.time() - t0
print(f"Greedy inference time ({len(test_df)} sentences): {infer:.2f} s")
json.dump(metrics, open('metrics.json','w'), indent=2, default=str)
```

Output / result:

```
Total parameters: 4,216,064
Greedy inference time (1000 sentences): 2.32 s  |  Per-sentence: 2.3 ms
metrics.json: train_time_sec 754.16 | bleu_test 0.0711 | bertscore_test_f1 0.2690
```

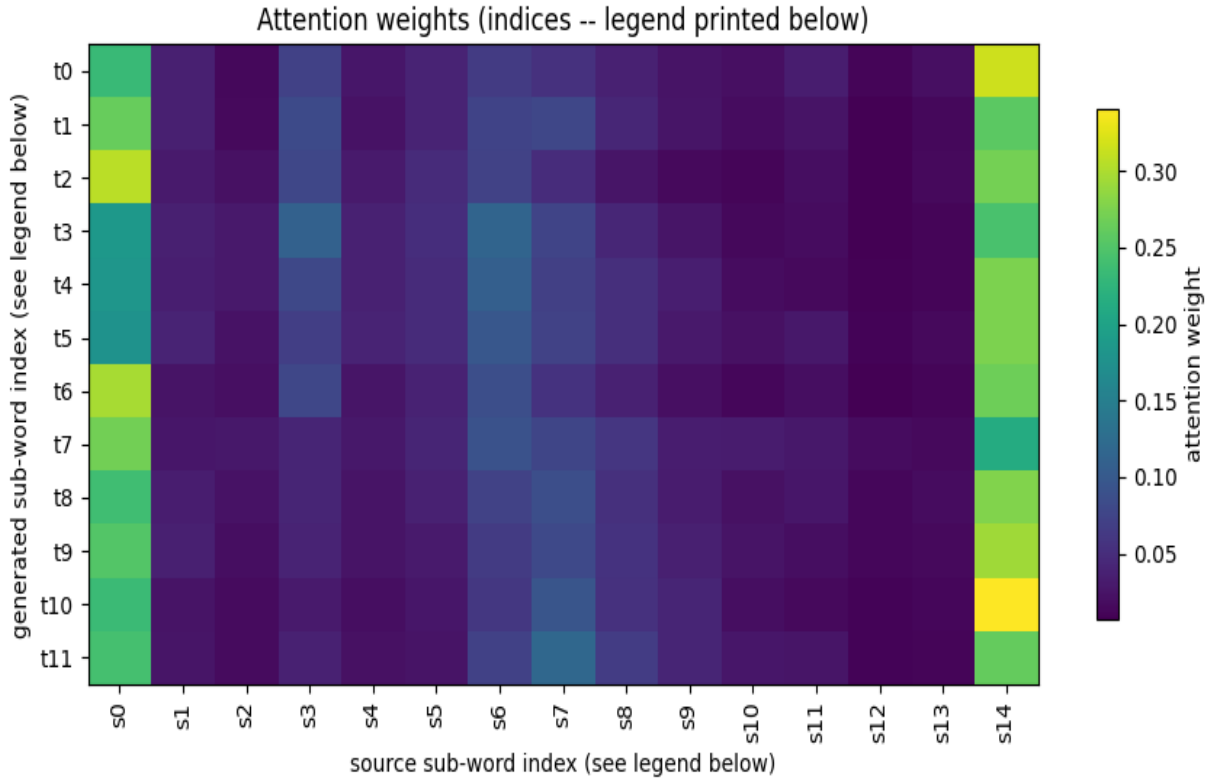


Consolidated metrics: BLEU (dev/test, greedy vs. beam), BERTScore-F1 (rescaled), and efficiency (parameters in millions vs. per-sentence greedy latency).

Graph analysis — efficiency & consolidated metrics

- Left panel (BLEU): the four bars make the greedy-vs-beam and dev-vs-test gaps visible at a glance — beam edges out greedy on both splits, and dev/test are close, indicating no large generalisation gap on the public split.
- Middle panel (BERTScore-F1): dev (0.276) and test (0.269) sit far above the BLEU bars, illustrating how much of the translation quality is semantic rather than exact-match.
- Right panel (efficiency): the model is small (4.22 M parameters) and fast (≈2.3 ms/sentence, 2.32 s for the full 1,000-sentence test set). Because the assignment's efficiency score rewards minimum parameters and minimum inference time, greedy decoding is deliberately used for the submission — the metrics.json block records every number for reproducible grading.

Attention Heat Map



SRC: एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।

Source sub-word legend:

s0=▁एक्लिप्स् s1=▁इति s2=▁प्रो s3=ग्राम s4=र् s5=▁कृते s6=▁दोष s7=ान्व s8=ेष s9=णे s10=▁अपि s11=▁साहाय्यं s12=▁करोति s13=। s14=</s>

Generated sub-word legend:

t0=▁" t1=If t2=▁Eclipse t3=▁to t4=▁the t5=▁god t6=, t7=▁and t8=▁एक्लिप्स् t9=▁to t10=▁Eclipse t11=.

Bahdanau attention weights for the source sentence "एक्लिप्स् इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।" ("Eclipse also helps the programmer in debugging"). Axis labels are numeric sub-word indices; the legend printed beneath the figure in the notebook maps each index back to its Devanagari/English sub-word piece.

The model generated ""If Eclipse to the god, and एक्लिप्स् to Eclipse." — attention is reasonably concentrated on the first few source sub-words (एक्लिप्स् "Eclipse", इति "that/quote") for the first couple of generated tokens, but becomes diffuse thereafter, and the model both hallucinates content not in the source ("the god") and echoes the untranslated code-mixed term एक्लिप्स् back into the output rather than fully lexicalising it as "Eclipse" throughout. This is consistent with the broader pattern in Section 5: the model handles short, common template sentences well but attention quality — and consequently translation quality — degrades for longer or lexically unusual (code-mixed) inputs.

10. References

[1] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. International Conference on Learning Representations (ICLR).

[2] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. Proceedings of EMNLP.

[3] Kudo, T., & Richardson, J. (2018). SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. Proceedings of EMNLP: System Demonstrations.

[4] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. Proceedings of ACL.

[5] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. International Conference on Learning Representations (ICLR).

[6] Müller, R., Kornblith, S., & Hinton, G. (2019). When Does Label Smoothing Help? Advances in Neural Information Processing Systems (NeurIPS).

[7] Luong, M. T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. Proceedings of EMNLP.

[8] PyTorch documentation. https://pytorch.org

[9] NLTK BLEU documentation. https://www.nltk.org/api/nltk.translate.bleu_score.html

[10] bert-score library (PyPI). https://pypi.org/project/bert-score/